

Reviewer Pack — Operator Norm Gap in Read-Twice BPs for IP_2

Entry a3b44525771f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 05:42:41 UTC

Operator Norm Gap in Read-Twice BPs for IP_2

Entry ID: a3b44525771f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 05:42:41 UTC

1. Conjecture

Field A (mathematical branch): Operator Space Theory **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For every read-twice branching program P computing IP_2 on n bits, the maximum singular value of the matrix M_P constructed from P 's transition matrices is $\Omega(n)$, whereas for read-once BPs, it is $O(\log n)$.

Rationale (proposer's reasoning):

The operator norm captures the 'spread' of the BP's computation, and the structure of read-twice BPs introduces dependencies that amplify this norm, creating a gap detectable via matrix analysis.

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 972c06813b60f944

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda k: abs(A[k][i]))
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(n):
                if j != i:
                    factor = A[j][i] / A[i][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def max_singular_value(A):
        A = gaussian_elimination(A)
        singular_values = [abs(A[i][i]) for i in range(min(len(A), len(A[0])))]
        return max(singular_values)

    def generate_read_twice_bp(n):
        # Simplified model of a read-twice branching program
        bp = []
        for _ in range(2 * n):
            bp.append(random.choice([0, 1]))
        return bp

    def construct_transition_matrix(bp, n):
        M = [[0] * (n + 1) for _ in range(n + 1)]
        state = 0
        for bit in bp:
            if bit == 0:
                state = (state << 1) & ((1 << (n + 1)) - 1)
            else:
                state = ((state << 1) | 1) & ((1 << (n + 1)) - 1)
            M[state][state] += 1
        return M

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
```

```

    bp = generate_read_twice_bp(n)
    M = construct_transition_matrix(bp, n)
    singular_value = max_singular_value(M)
    results.append({
        "n": n,
        "singular_value": singular_value
    })

metric_name = "max_singular_value"
metric_value = sum(result["singular_value"] for result in results) / len(results)
instances_tested = len(results)
conjecture_holds = all(result["singular_value"] >= n for result in results)
counterexample = "" if conjecture_holds else f"n={results[0]['n']}, singular_value={results[0]['singular_value']}"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{result['counterexample']}' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6bc758db.py", line 87, in <module>
    trial_result = run_trial(seed)
    ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6bc758db.py", line 61, in run_trial

```

```
M = construct_transition_matrix(bp, n)
~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6bc758db.py", line 53, in construct_tra
M[state][state] += 1
~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before producing results, preventing evaluation of conjecture | next: Debug the construct_transition_matrix implementation to fix index out of range error

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	78379
2	preregistration	ollama_remote	qwen3:8b	0	0	31493
3	novelty	ollama_remote	qwen3:8b	0	0	27460
4	novelty	ollama_remote	qwen3:8b	0	0	23190
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29034
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10620
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41730
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26734
9	judge	ollama_remote	qwen3:8b	0	0	120795

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 389436 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/a3b44525771f.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/a3b44525771f.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/a3b44525771f.tar.gz* (if generated)